Web Engineering & Development (SWE 363)

MongoDB & Mongoose

In today's lecture:

- MongoDB: documents, collections, and the MongoDB Shell
- CRUD operations in the shell
- Mongoose: schemas, models, and CRUD in Node.js

Reference:

- Zybook: 7.1, 7.2

Activity: Think-Pair-Share (2 min)

Recall 6.1: a new product with an extra color field is added to both databases.

```
INSERT INTO Product VALUES (999, 315.99, 'Smart watch', 'blue');
```

```
db.products.insertOne({ _id: 999, price: 315.99, title: "Smart watch", color: "blue" })
```

1. A relational database stores data in _____; a document database stores documents in _____ format.
2. Why does the SQL statement fail while the MongoDB command works?
3. What could a table, a row, and a column be called in MongoDB?

7.2 MongoDB

What is MongoDB?

> **MongoDB** stores data as **documents** (field/value pairs, like JSON), grouped into **collections**

- **Why:** Node.js works seamlessly with MongoDB: JSON data, JavaScript queries
- **Field/value pair:** `gpa: 3.1` → field `gpa`, value `3.1`

```
mydb database
  → students collection
    → { name: "Sue", gpa: 3.1, interests: ["biking", "reading"] }
    → { name: "Larry", gpa: 2.5, interests: ["RPGs", "chess"] }
    → { name: "Anne", gpa: 4.0, interests: ["coding", "Pilates"] }
```

MongoDB Shell (mongosh)

> **MongoDB Shell** is a command-line interface for MongoDB; the `mongosh` command starts it

- **Why:** create, query, and delete documents by typing commands

```
$ mongosh
test> use mydb              // creates or selects the "mydb" database
switched to db mydb
mydb> show dbs              // lists all databases
mydb> show collections      // lists all collections in mydb
mydb> exit                  // quits the shell
```

Activity: Classify, then name the command

```
test> use dealer
dealer> db.autos.insertOne({ _id: 200, make: "Ford", model: "Mustang" })
```

Database, collection, document, field, or value?

1. `dealer`
2. `autos`
3. `{ _id: 200, make: "Ford", model: "Mustang" }`
4. `make`
5. `"Ford"`

Inserting Documents

> `insertOne()` inserts a single document into a collection

- **Why:** this is how data gets into MongoDB (the C in CRUD)
- Every document gets a unique `_id`, the **primary key**

```
mydb> db.students.insertOne({ name: "Sue", gpa: 3.1 })
{ acknowledged: true, insertedId: ObjectId("62794229fc4ebd4933877a9d") }
```

Finding Documents

> `find()` returns the documents that match a query; `findOne()` returns only the first

- **Why:** queries read just the data you need
- **Query operators** compare values: `$gt` means greater than

```
db.students.find()                          // all students
db.students.find({ name: "Sue" })           // students named Sue
db.students.find({ gpa: { $gt: 3.5 } })     // students with gpa > 3.5
db.students.findOne({ gpa: { $gt: 3.5 } })  // only the first match
```

Updating & Deleting Documents

> `updateOne()` changes the first matching document; `deleteOne()` removes it

- **Why:** keep data current and remove what is no longer needed
- `updateMany()` / `deleteMany()` → all matching documents

```
mydb> db.students.updateOne({ name: 'Sue' }, { $set: { gpa: 3.3 } })
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }
mydb> db.students.deleteOne({ gpa: { $lt: 3.5 } })
{ acknowledged: true, deletedCount: 1 }
```

Activity: Predict: which documents are returned (1–3)? What changes (4–5)?

_id	make	year	price
100	Ford	2014	13500
200	Honda	2013	16900
300	Dodge	2012	10800
400	Toyota	2013	13400

Key: `$gt` >, `$gte` ≥, `$lt` <, `$set` sets a field

```
db.autos.find({ year: { $gte: 2013 } })                                  // 1.
db.autos.findOne({ year: { $gte: 2013 } })                               // 2.
db.autos.find({ price: { $lt: 13500 } })                                 // 3.
db.autos.updateOne({ price: { $gt: 13000 } }, { $set: { sold: true } })  // 4.
db.autos.deleteOne({ year: 2013 })                                       // 5.
```

7.1 Mongoose

What is Mongoose?

> The **mongoose** module adds **structured schemas** to MongoDB collections in Node.js

- **Why:** documents follow a defined structure, and some operations get simpler
- `connect()` must run before any database operation

```
// npm install mongoose
const mongoose = require("mongoose");
mongoose.connect("mongodb://localhost/mydb");   // creates mydb if it doesn't exist
```

Schemas & Models

> A **schema** defines the structure of documents; a **model** is a constructor compiled from a schema

- **Why:** every student needs a `name`; `gpa` must be 0 to 4
- Model `"Student"` → collection **"students"** (the plural)

```
const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  gpa: { type: Number, min: 0, max: 4 },
  birthDate: { type: Date, default: Date.now }
});
const Student = mongoose.model("Student", studentSchema);
```

Saving Documents

> `save()` stores a model instance (a **document**) in the database

- **Why:** `new Student(...)` alone does not touch the database
- `await` waits for the save to finish; the document then has an `_id`

```
app.get("/create", async function(req, res) {
  const stu = new Student({ name: "Sue Black", gpa: 3.1 });
  await stu.save();
  res.send(`Saved ${stu.name}.`);
});
```

Activity: What happens if… (each change on its own)

```
mongoose.connect("mongodb://localhost/mydb");

const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  birthDate: { type: Date, default: Date.now }
});
// Model "Student" → collection "students"
const Student = mongoose.model("Student",
  studentSchema);

app.get("/create", async function(req, res) {
  const stu = new Student({ name: "Sue Black" });
  await stu.save();
  res.send(`Saved ${stu.name}.`);
});
```

1. A student is saved with no `birthDate`?
2. A student is saved with no `name`?
3. `mongoose.model("Dog", dogSchema)`: which collection?
4. The URL ends in `/students` instead of `/mydb`?
5. `await` is removed before `stu.save()`?

CRUD with Mongoose

> Model methods work like the shell's `find()`, `findOne()`, `updateOne()`, and `deleteOne()`

- **Why:** same queries and operators, now inside Node.js (use `await`)

```
const students = await Student.find({ gpa: { $gte: 3 } });
const stu = await Student.findOne({ name: "Sue Black" });

const result = await Student.updateOne({ name: "Sue Black" }, { $set: { gpa: 3.9 } });
console.log("Docs updated: " + result.modifiedCount);

const result2 = await Student.deleteOne({ name: "Sue Black" });
console.log("Docs deleted: " + result2.deletedCount);
```

Activity: Predict the result (each line on its own)

name	Sue	Mara	Dana	Li
gpa	3.1	2.6	3.3	4.0

```
await Student.find({ gpa: { $gte: 3.2 } });                          // 1. which students?
await Student.findOne({ gpa: { $lt: 3.5 } });                        // 2. which student?
await Student.updateOne({ gpa: { $gt: 3 } }, { $set: { gpa: 4 } }); // 3. who changes?
await Student.deleteOne({ name: "Greg" });                           // 4. deletedCount?
```

Key: `$gt` >, `$gte` ≥, `$lt` <, `$set` sets a field

Activity: Exit ticket - find the 5 bugs

```
const mongoose = require("mongoose");
mongoose.connect("http://localhost/mydb");

const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  gpa: { type: Number, min: 0, max: 4 }
});
const Student = mongoose.model("Student", studentSchema);

async function addStudent() {
  const stu = new Student({ gpa: 3.5 });
  stu.save();
  const honors = await Student.find({ gpa: { gte: 3.5 } });
  await Student.deleteOne(123);
}
```

Key Takeaways

- MongoDB stores **documents** (field/value pairs) in **collections** inside databases
- **mongosh:** `use mydb`, `show dbs`, `show collections`
- **Shell CRUD:** `insertOne()`, `find()`, `updateOne()` with `$set`, `deleteOne()`
- **Query operators** like `$gt` and `$lt` filter documents
- **Mongoose:** `connect()` → schema → model → document → `save()`
- **Mongoose CRUD:** the same methods, called on the model with `await`

30m

Demo

7.1 MongoDB

MongoDB & Mongoose hands-on:

- MongoDB and mongosh setup
- CRUD operations in mongosh
- Mongoose connection, schema, and CRUD

Next Class

- RESTful web APIs (7.3)
- Using RESTful web APIs with Fetch (7.5)